

Carátula

Institución: UNIVERSIDAD TECNOLÓGICA NACIONAL - FACULTAD REGIONAL SAN NICOLÁS

Carrera: TECNICATURA UNIVERSITARIA EN PROGRAMACIÓN

Materia: Programación 1

Título del proyecto: Gestión de Datos de Países en Python: filtros, ordenamientos y estadísticas

Integrantes: Enzo Villegas, Cintia Ledesma

Fecha de entrega: 27 de junio de 2026

Índice

1. [Marco Teórico](#)
 1. [Listas](#)
 2. [Diccionarios](#)
 3. [Funciones](#)
 4. [Condicionales](#)
 5. [Estructuras repetitivas](#)
 6. [Ordenamientos](#)
 7. [Estadísticas básicas](#)
 8. [Archivos CSV](#)
 2. [Decisiones Técnicas y Arquitectura](#)
 1. [Estructura general del programa](#)
 2. [Diagrama de flujo](#)
 3. [Capturas de pantalla](#)
 2. [Dificultades y Conclusiones](#)
 1. [Dificultades encontradas](#)
 2. [Conclusiones](#)
 2. [Bibliografía / Webgrafía](#)
 3. [Enlaces](#)
-

1. Marco Teórico

1.1 Listas

Una lista en Python es una estructura de datos que permite almacenar múltiples elementos en una secuencia ordenada. Se definen con corchetes (`[]`) y pueden contener elementos de cualquier tipo. Las listas son mutables, lo que significa que sus elementos se pueden modificar, agregar o eliminar después de su creación.

En este proyecto, la lista principal es `dataset`, que almacena todos los países cargados. Cada elemento de la lista es un diccionario que representa un país. Se utiliza el método `append()` para agregar nuevos países a la lista, y se recorren los elementos con bucles `for` para realizar búsquedas, filtros y cálculos.

Ejemplo de uso en el proyecto:

```
dataset = []
```

```
dataset.append({"nombre": "Argentina", "poblacion": 45376763, ...})
```

1.2 Diccionarios

Un diccionario en Python es una estructura de datos que almacena pares de clave-valor. Se definen con llaves (`{}`) y permiten acceder a los valores de forma directa usando su clave. Al igual que las listas, los diccionarios son mutables.

En este proyecto, cada país se representa como un diccionario con cuatro claves: `nombre`, `poblacion`, `superficie` y `continente`. Esta estructura permite acceder a cada dato del país de forma clara y legible. También se utiliza un diccionario en la función `obtener_paises_por_continente()` para contar la cantidad de países por continente.

Ejemplo de uso en el proyecto:

```
pais = {
```

```
    "nombre": "Argentina",
```

```
    "poblacion": 45376763,
```

```
    "superficie": 2780400,
```

```
"continente": "América"

}

print(pais["nombre"]) # Argentina
```

1.3 Funciones

Una función es un bloque de código reutilizable que realiza una tarea específica. Se definen con la palabra clave `def` y pueden recibir parámetros y devolver valores con `return`. Las funciones permiten modularizar el código, haciendo que sea más legible, organizado y fácil de mantener.

En este proyecto se sigue el principio de que cada función tiene una única responsabilidad. Por ejemplo, `pedir_numero()` se encarga exclusivamente de solicitar y validar un número entero, `mostrar_pais()` se encarga de imprimir los datos de un país en un formato legible, y `filtrar_por_continente()` se encarga de filtrar y mostrar los países de un continente dado.

1.4 Condicionales

Las estructuras condicionales permiten ejecutar diferentes bloques de código según se cumpla o no una condición. En Python se implementan con las palabras clave `if`, `elif` y `else`.

En este proyecto se utilizan condicionales en múltiples contextos:

- En el menú principal, para determinar qué acción ejecutar según la opción seleccionada por el usuario.
- En las validaciones, para verificar que los datos ingresados sean correctos (por ejemplo, que un número no sea negativo).
- En las búsquedas y filtros, para comparar los datos de cada país contra los criterios ingresados.

1.5 Estructuras repetitivas

Las estructuras repetitivas (o bucles) permiten ejecutar un bloque de código de forma reiterada. Python ofrece dos tipos: `while` (repite mientras se cumpla una condición) y `for` (itera sobre una secuencia de elementos).

En este proyecto se usan ambos tipos:

- **while:** se utiliza en el menú principal para mantener el programa en ejecución hasta que el usuario elija salir, y en las funciones de validación para seguir pidiendo datos hasta que sean válidos.
- **for:** se utiliza para recorrer la lista de países al buscar, filtrar, ordenar y calcular estadísticas.

1.6 Ordenamientos

Ordenar datos significa reorganizar los elementos de una colección según un criterio determinado. Python ofrece la función integrada `sorted()`, que devuelve una nueva lista con los elementos ordenados sin modificar la lista original. Esta función acepta un parámetro `key` para indicar el criterio de ordenamiento y un parámetro `reverse` para definir el sentido (ascendente o descendente).

En este proyecto se utiliza `sorted()` para ordenar los países por nombre, población o superficie. Se define una función interna `obtener_criterio()` que extrae el valor del campo por el cual se desea ordenar.

Ejemplo de uso en el proyecto:

```
países_ordenados = sorted(dataset, key=obtener_criterio, reverse=desc)
```

1.7 Estadísticas básicas

Las estadísticas básicas permiten obtener indicadores clave a partir de un conjunto de datos. Las más comunes son el valor máximo, el valor mínimo y el promedio (media aritmética).

En este proyecto se calculan las siguientes estadísticas:

- **País con mayor y menor población:** se recorre la lista comparando la población de cada país para encontrar el máximo y el mínimo.
- **Promedio de población y superficie:** se suman todos los valores del criterio y se dividen por la cantidad total de países.
- **Cantidad de países por continente:** se utiliza un diccionario como acumulador, donde cada clave es un continente y el valor es la cantidad de países que pertenecen a él.

1.8 Archivos CSV

CSV (Comma-Separated Values) es un formato de archivo de texto plano donde los datos se organizan en filas y columnas, separados por comas. Es un formato ampliamente utilizado para el intercambio de datos por su simplicidad y compatibilidad.

En este proyecto, el archivo `dataset.csv` contiene la información base de los países. El programa lo lee línea por línea, separa los valores usando el método `split(",")` y convierte los datos numéricos a sus tipos correspondientes (`int` para población, `float` para superficie). La primera línea del archivo (encabezado) se omite durante la lectura.

2. Decisiones Técnicas y Arquitectura

2.1 Estructura general del programa

El programa se organiza de la siguiente manera:

- **Funciones de entrada de datos:** `pedir_numero()`, `pedir_numero_limite()`, `pedir_palabra()`, `pedir_pais_existente()`, `pedir_nuevo_pais()`. Se encargan de solicitar datos al usuario con las validaciones necesarias.
- **Funciones de manipulación del dataset:** `cargar_dataset()`, `agregar_pais()`, `actualizar_poblacion_superficie()`. Permiten cargar, agregar y modificar datos.
- **Funciones de consulta:** `buscar_pais()`, `filtrar_paises()`, `ordenar_paises()`. Permiten buscar, filtrar y ordenar los países según distintos criterios.
- **Funciones de estadísticas:** `obtener_pais_mayor_poblacion()`, `obtener_pais_menor_poblacion()`, `obtener_promedio_criterio()`, `obtener_paises_por_continente()`, `mostrar_estadisticas()`. Calculan y muestran indicadores clave.
- **Funciones auxiliares:** `limpiar_nombre()`, `mostrar_pais()`, `pais_existe()`, `mostrar_menu()`. Resuelven tareas de soporte.
- **Programa principal:** un bucle `while` que muestra el menú y ejecuta la opción elegida.

Se tomó la decisión de representar cada país como un diccionario dentro de una lista, ya que permite acceder a los datos por nombre de campo (en lugar de por posición), lo que hace el código más legible.

La función `limpiar_nombre()` normaliza los textos eliminando acentos y convirtiendo a minúsculas, lo que permite que las búsquedas y filtros funcionen correctamente sin importar cómo el usuario escriba los nombres.

2.2 Diagramas de flujo

Diagrama 1: Menú principal

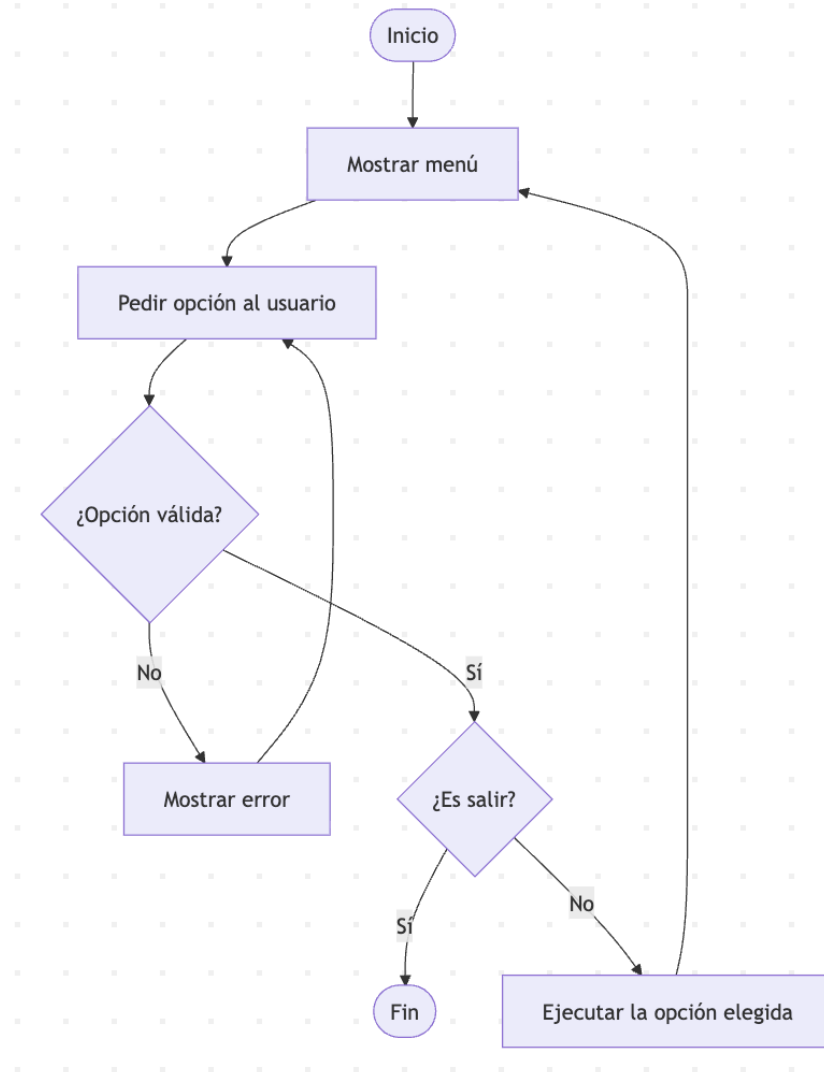


Diagrama 2: Agregar país

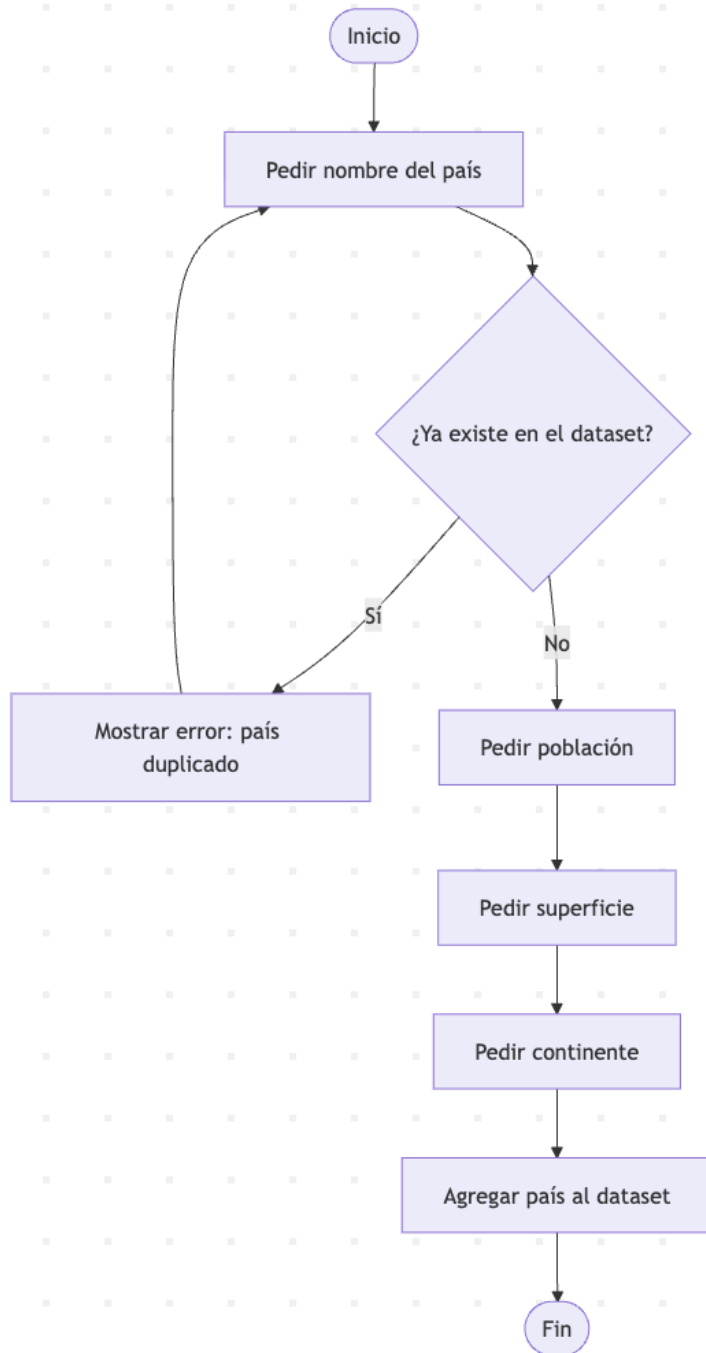
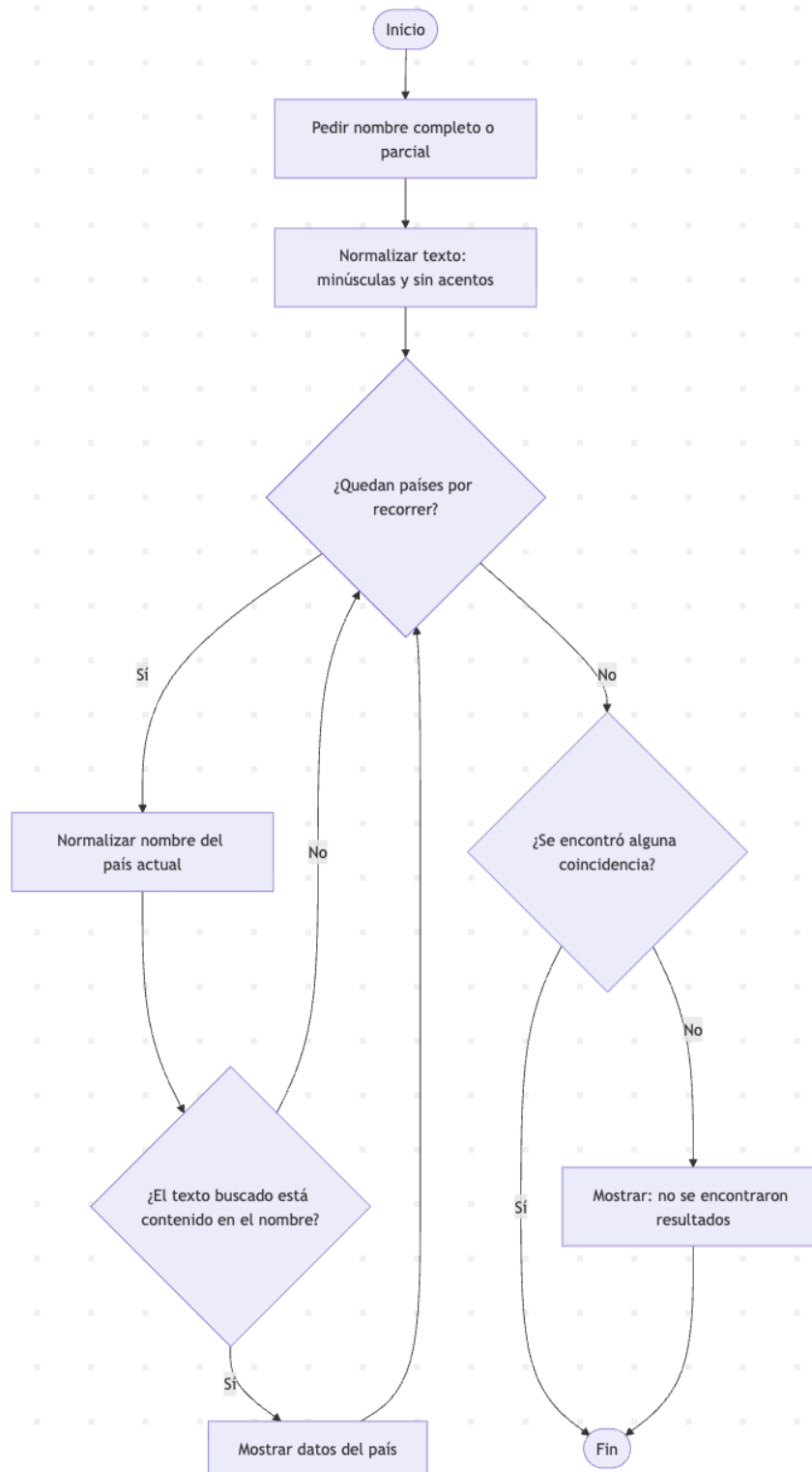


Diagrama 3: Buscar país



2.3 Capturas de pantalla

```
integrador $ python integrador.py
1. cargar dataset
2. mostrar dataset
3. agregar pais
4. actualizar pais
5. buscar pais
6. filtrar paises
7. ordenar paises
8. mostrar estadisticas
9. salir
Ingrese una opcion: 1
Se cargaron 4 paises exitosamente.
```

```
Se cargaron 4 paises exitosamente.
1. cargar dataset
2. mostrar dataset
3. agregar pais
4. actualizar pais
5. buscar pais
6. filtrar paises
7. ordenar paises
8. mostrar estadisticas
9. salir
Ingrese una opcion: 2
Argentina (Poblacion = 45376763, Superficie = 2780400.0, Continente = América)
Japón (Poblacion = 125800000, Superficie = 377975.0, Continente = Asia)
Brasil (Poblacion = 213993437, Superficie = 8515767.0, Continente = América)
Alemania (Poblacion = 83149300, Superficie = 357022.0, Continente = Europa)
```

```
1. cargar dataset
2. mostrar dataset
3. agregar pais
4. actualizar pais
5. buscar pais
6. filtrar paises
7. ordenar paises
8. mostrar estadisticas
9. salir
Ingrese una opcion: 5
Ingrese el nombre completo o parcial del pais: gent
Argentina (Poblacion = 45376763, Superficie = 2780400.0, Continente = América)
```

3. Dificultades y Conclusiones

3.1 Dificultades encontradas

Durante el desarrollo del proyecto surgieron los siguientes obstáculos técnicos:

- **Manejo de acentos en las búsquedas:** al buscar o filtrar países, las tildes provocaban que no se encontraran coincidencias (por ejemplo, buscar "America" no encontraba "América"). Esto se resolvió creando la función `limpiar_nombre()` que utiliza el método `replace()` de strings para reemplazar las vocales acentuadas por sus equivalentes sin acento antes de comparar.
- **Ordenamiento por diferentes criterios:** fue necesario investigar cómo funciona la función `sorted()` de Python y su parámetro `key` para poder ordenar la lista de diccionarios por distintos campos. Se resolvió creando una función interna que extrae el valor del campo deseado.
- **Validación de entradas:** asegurar que el programa no falle ante entradas inválidas (textos en lugar de números, campos vacíos, opciones fuera de rango) requirió implementar funciones específicas de validación con bucles `while` y bloques `try/except`.

3.2 Conclusiones

Este trabajo permitió aplicar de forma integrada los conceptos fundamentales aprendidos en la materia Programación 1. La construcción de un sistema completo, desde la lectura de datos

hasta la generación de estadísticas, nos permitió entender cómo se complementan las distintas estructuras de datos y de control para resolver un problema real.

La modularización del código en funciones con responsabilidades claras facilitó tanto el desarrollo como la depuración de errores. Aprendimos que un buen diseño de funciones permite reutilizar código y hacer modificaciones sin afectar otras partes del programa.

Consideramos que el resultado final cumple con todos los requerimientos planteados y refleja el aprendizaje adquirido durante la cursada.

4. Bibliografía / Webgrafía

- Python Software Foundation. (s.f.). *The Python Tutorial*. Documentación oficial de Python. <https://docs.python.org/3/tutorial/>
 - Python Software Foundation. (s.f.). *Built-in Functions: sorted()*. Documentación oficial de Python. <https://docs.python.org/3/library/functions.html#sorted>
 - W3Schools. (s.f.). *Python sorted() Function*. https://www.w3schools.com/python/ref_func_sorted.asp
 - W3Schools. (s.f.). *Python String replace() Method*. https://www.w3schools.com/python/ref_string_replace.asp
 - Python Software Foundation. (s.f.). *Reading and Writing Files*. Documentación oficial de Python. <https://docs.python.org/3/tutorial/inputoutput.html#reading-and-writing-files>
-

5. Enlaces

- Repositorio GitHub: [CintiaHL/Trabajo-Integrador-Enzo-Villegas-Cintia-Ledesma](#)
- Video demostrativo: <https://youtu.be/e3Wu8Dh6jKo>